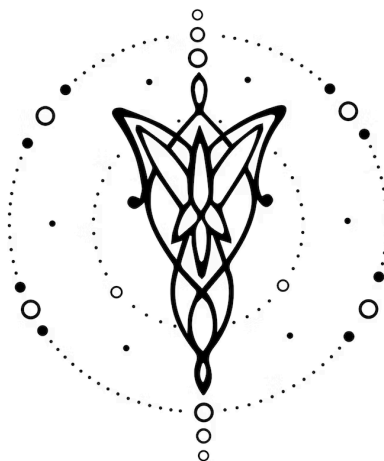




Especificación Murrur

Diseño e Implementación de un Lenguaje

6 de septiembre de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombres	Apellidos	Legajos	E-mails
Benicio	Leite	64181	bleite@itba.edu.ar
Juan	Humphreys	65477	jhumphreys@itba.edu.ar
Bautista	Etcheverry	65765	betcheverry@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/BautiEtcheverry/murrur>

3. Dominio

Desarrollar un lenguaje que permita especificar y simular misiones de enjambres de drones, compuestas por aeronaves individuales agrupadas en flotas que operan sobre zonas delimitadas del espacio. El lenguaje deberá permitir declarar los tipos de dron y sus capacidades, como autonomía, velocidad, sensores y alcance de enlace. Además, componer flotas a partir de ellos, delimitar el espacio de vuelo y sus puntos de interés, y describir las maniobras que cada grupo ejecuta a lo largo del tiempo, como despegues, barridos, formaciones y retornos a base o zona de aterrizaje, coordinando las aeronaves en simultáneo o en secuencia sobre una ventana temporal determinada.

El espacio se modelará como un plano con eje vertical y toda magnitud física se expresa con unidades explícitas (metros, segundos, grados, entre otras). Para reducir la complejidad del proyecto, las trayectorias no se resolverán mediante un modelo cinemático continuo, sino que serán aproximadas a través de un muestreo temporal discreto: el compilador reconstruye la posición de cada aeronave en instantes sucesivos, y es sobre ese muestreo que evalúa las restricciones de la misión.

Toda la simulación se realiza dentro de la memoria del mismo programa; las aeronaves, las zonas y las trayectorias son completamente virtuales y solo existen dentro de la aplicación. No existe conexión con drones reales, telemetría, GPS ni controladores de vuelo físicos, así como tampoco una simulación aerodinámica, de viento o de perturbaciones del mundo real. El modelo cinemático es una aproximación suficiente para planificar y validar una misión, pero no para pilotarla.

La implementación satisfactoria de este lenguaje permitiría diseñar, versionar y validar misiones multi-dron como si fueran código, detectando errores de cobertura, separación entre aeronaves, autonomía insuficiente o pérdida de enlace. De esta manera, la responsabilidad de la seguridad se traslada del operador al compilador, minimizando los riesgos y los costos de descubrir dichas fallas en pleno vuelo.

4. Construcciones

Murmur es un lenguaje declarativo para especificar misiones de enjambre de drones. El compilador toma la descripción de las aeronaves pertenecientes a una flota, recibe los parámetros e instrucciones de la misión que se quiere analizar, reconstruye las trayectorias, verifica que la misión sea segura y recién entonces emite los artefactos finales.

Las secciones a continuación describen las diferentes construcciones del lenguaje siguiendo un orden específico. Primero las magnitudes con las que se expresa todo lo demás, después las declaraciones que definen los recursos y el espacio, luego la misión propiamente dicha con sus maniobras y su composición temporal, y finalmente las restricciones y el conjunto de verificaciones que el compilador aplica sobre el resultado.

4.1. Magnitudes y unidades

Las magnitudes físicas en el lenguaje se escriben con un formato específico, un valor numérico seguido de su unidad entre corchetes. Por ejemplo, si se quiere describir la velocidad de un dron se notaría, 18 [m/s]. La unidad es verificada en tiempo de compilación para asegurar que corresponde a la dimensión que el parámetro espera. Cabe aclarar que las unidades que comparten dimensión son intercambiables de modo que, 1800 [s] y 30 [min] son equivalentes en el lenguaje.

Las siguientes unidades son reconocidas:

- Tiempo: [s], [min].
- Longitud: [m], [km].
- Velocidad: [m/s], [km/h].
- Aceleración: $[m/s^2]$.
- Angulo: [deg], [rad].
- Resolución de sensores y porcentajes: [px], [%].

Los puntos del plano constituyen la única excepción a la regla de la unidad explícita. Se escriben como pares de coordenadas, (x, y), y se interpretan siempre en metros respecto del origen del sistema de referencia local de la misión. La componente vertical no forma parte del punto, la altitud se declara siempre como un parámetro explícito de la maniobra o estación que la requiera.

4.2. Aeronaves y flotas

Un programa empieza con la declaración de las aeronaves que van a tener parte en la misión. El tipo de dron agrupa las variables que determinan las capacidades y características de un tipo determinado de aeronave, se declaran mediante un bloque en el cual se enumeran las mismas.

```
drone_type Scout {  
    endurance 28 [min];  
    max_speed 18 [m/s];  
    max_accel 6 [m/s^2];  
    payload    rgb_camera resolution 4000 [px] fov 84 [deg];  
    link_range 1200 [m];  
}
```

Cada uno de estos campos es relevante porque habilita una verificación posterior. En primer lugar, la autonomía permite detectar misiones que agotan la batería antes del retorno. La velocidad y la aceleración máximas permiten rechazar maniobras físicamente imposibles, como una transición de formación que exigiría desplazarse cuarenta metros en dos segundos. El campo de visión del sensor, junto con la altitud de vuelo, determina el ancho de la franja que la aeronave observa en cada pasada, del cual depende el cálculo de cobertura. El alcance de enlace

permite detectar tramos de misión en los que una aeronave perdería contacto con su estación. Los campos `max_accel`, `payload` y `link_range` son opcionales, y su omisión simplemente desactiva las verificaciones que dependen de ellos.

Sobre los tipos declarados se componen las flotas, que son los conjuntos concretos de aeronaves que ejecutan la misión. Una flota puede ser homogénea o combinar varios tipos. Nótese un ejemplo de declaración de flota:

```
fleet Bravo = 6 of Scout + 2 of Thermal;
```

También al tener la posibilidad de que exista una flota heterogénea (Varios tipos de aeronaves) plantea la necesidad de dirigirse a una parte de ella. Con este fin el lenguaje permite referenciar subconjuntos por tipo mediante notación de punto. Por ejemplo en la declaración de arriba, Bravo denota las ocho aeronaves y Bravo.Scout denota únicamente las seis del tipo indicado. Internamente cada aeronave recibe un identificador estable de la forma Scout#3, qué es lo que aparece en los mensajes de error cuando alguna verificación falla.

4.3. El espacio de vuelo

Las zonas de vuelo delimitan las regiones del plano sobre las que la misión opera. Se declaran como polígonos, enumerando sus vértices, o como círculos, indicando centro y radio:

```
zone AOI = polygon [(0,0), (400,0), (400,300), (150,300)];
zone LKP = circle center (200, 180) radius 40 [m];
```

Una misma zona cumple roles distintos según dónde se la use. Si es referenciada por una maniobra de barrido es el área de trabajo que hay que cubrir. Pero si es declarada como exclusión dentro del bloque de restricciones es un volumen prohibido que ninguna trayectoria puede atravesar. Esta doble función evita introducir dos construcciones separadas para lo que geométricamente es lo mismo.

Es importante aclarar que los polígonos deben ser convexos y tener al menos tres vértices. La restricción es una decisión consciente de alcance. Generar un patrón de barrido sistemático sobre un polígono cóncavo requiere descomponerlo previamente en regiones convexas, un problema de geometría computacional que excede el alcance planteado en el trabajo práctico.

Las estaciones son los puntos desde los cuales las aeronaves despegan y a las cuales regresan. Se declaran indicando su posición y su altitud:

```
station Base = point (10, 10) altitude 0 [m];
```

4.4 Formaciones

Una formación es una disposición geométrica relativa que la flota adopta en vuelo. En nuestro lenguaje planteamos tres tipos. La grilla es una formación la cual organiza las aeronaves en filas y columnas separadas por un espacio específico. Luego tenemos el círculo, que las distribuye con una separación equivalente a todas las aeronaves sobre una circunferencia. Por último, la línea que las alinea sobre un eje de avance. La declaración de la formación:

```
formation Peine = line spacing 30 [m];
formation Grilla = grid 3 x 2 spacing 15 [m];
formation Anillo = circle radius 25 [m];
```

Las formaciones pueden declararse con nombre, cuando se las utiliza varias veces a lo largo de la misión, o directamente en el punto de uso cuando se emplean una sola vez. Las posiciones que genera una formación se interpretan siempre respecto del centro de la flota, de modo que la formación describe la figura y no su ubicación absoluta.

A diferencia del círculo y la línea, que se acomodan a la cantidad de aeronaves que los adopten, la grilla fija su cardinalidad en la declaración. Si la flota que la adopta tiene una cantidad distinta, el programa se rechaza.

4.5. La misión

La misión es el único bloque ejecutable del lenguaje y agrupa todas las maniobras bajo una ventana temporal explícita:

```
mission SAR window 0 [s] .. 900 [s] { ... }
```

La ventana de tiempo define un intervalo dentro del cual toda maniobra debe haber finalizado y el compilador rechaza cualquier programa que la exceda. Su función es obligar al programador a declarar por adelantado cuánto tiempo requiere, en lugar de descubrir la duración total como consecuencia accidental de las maniobras que fue escribiendo.

El cuerpo de la misión se comporta como una secuencia implícita. Salvo que una maniobra fije un instante de inicio específico, cada sentencia comienza cuando termina la anterior. Esto permite escribir misiones simples sin necesidad de bloques de composición explícitos.

Cada maniobra tiene un instante de inicio y una duración, que se determinan de tres formas distintas. Algunas maniobras fijan su inicio de manera absoluta mediante la cláusula “at”, como ocurre con el despegue y el aterrizaje. Otras declaran explícitamente cuánto duran mediante “over” o “for”, como por ejemplo, el cambio de formación, la órbita y la espera. Y el barrido no declara ninguna de las dos cosas, su duración la calcula el compilador, a partir del largo del recorrido que el patrón genera sobre la zona y de la velocidad de operación de las

aeronaves. Este último caso es el que mejor ilustra qué gana el operador al usar el lenguaje. Al escribir un barrido, el programador enuncia qué zona quiere cubrir, con qué patrón, con qué espaciado y a qué altitud. No calcula cuántas pasadas hacen falta, ni dónde están los puntos de giro, ni cuánto tiempo requiere el recorrido. Todo eso lo resuelve el compilador.

4.6. Maniobras

Las maniobras analizadas en el lenguaje abarcan los movimientos esenciales para el funcionamiento de la flota. Para empezar tenemos el despegue y el aterrizaje cuya función es llevar la flota de la estación hacia la altura de trabajo y viceversa.

Bravo takeoff from Base at 0 [s];
Bravo land at Base at 850 [s];

Otra maniobra disponible es el cambio de formación, que reubica a las aeronaves desde su disposición actual hasta la formación destino. Cada dron se desplaza en línea recta y a velocidad constante, completando el trayecto en el intervalo indicado:

Bravo.Scout morph to Peine over 25 [s];

Luego está el barrido, está consiste en recorrer una zona a altitud constante según uno de tres patrones. El patrón lawnmower traza pasadas paralelas separadas por el espacio indicado, con giros en los extremos. El patrón spiral recorre la zona desde el perímetro hacia el centro. Y por último, el patrón perimeter sigue el borde.

Cuando la zona a barrer intersecta una zona excluida por el bloque de restricciones, el patrón omite los tramos de pasada que la atraviesan, la aeronave interrumpe la pasada al alcanzar el borde de la exclusión y la retoma del otro lado, contorneándola.

Bravo.Scout sweep AOI with lawnmower spacing 25 [m] altitude 60 [m];
Bravo.Scout sweep Lote with spiral spacing 60 [m] altitude 100 [m];
Linea sweep Traza with perimeter spacing 10 [m] altitude 35 [m];

La órbita es la maniobra que permite mantener a la flota girando alrededor del centro de una zona durante un intervalo dado, y la espera la deja en vuelo estacionario en su posición actual:

Bravo.Thermal orbit LKP radius 70 [m] altitude 90 [m] for 300 [s];
Bravo.Thermal hold for 45 [s];

Por último, el retorno lleva a todas las aeronaves de vuelta a una estación. Su cláusula before no indica cuándo empieza sino cuándo debe haber terminado, y actúa por lo tanto como

una fecha límite contra la cual el compilador verifica el consumo energético de la misión completa:

all return to Base before 880 [s];

4.7. Composición temporal

Murmur no tiene condicionales ni bucles. Su flujo de control es temporal, esto significa que las construcciones de composición no deciden qué se ejecuta, sino cuándo y en simultáneo con qué.

El lenguaje distingue dos bloques. El primero es el bloque secuencial el cual encadena sus sentencias, de modo que cada una comienza en el instante en que termina la anterior. Luego el bloque paralelo arranca todas sus sentencias en el mismo instante y termina cuando concluye la más larga de ellas.

```
parallel {
  sequence {
    Bravo.Scout morph to Peine over 25 [s];
    Bravo.Scout sweep AOI with lawnmower spacing 25 [m] altitude 60 [m];}

  sequence {
    Bravo.Thermal orbit LKP radius 70 [m] altitude 90 [m] for 300 [s];
    Bravo.Thermal hold for 60 [s]; }
}
```

Ambos bloques son a su vez sentencias con su propio instante de inicio y su propia duración, lo que permite anidarlos libremente. La combinación más frecuente es un bloque secuencial dentro de uno paralelo, que expresa la situación habitual de dos subflotas trabajando al mismo tiempo, cada una con su propia secuencia de maniobras.

El paralelismo introduce una condición que el compilador verifica. Esta es que la misma flota no puede aparecer en dos sentencias del mismo bloque paralelo, porque una aeronave no puede ejecutar dos maniobras en simultáneo. En cambio, dos subflotas de una misma flota sí pueden hacerlo, y de hecho ese es el uso previsto de la notación de punto.

4.8. Restricciones

El bloque de restricciones es la única construcción del lenguaje que no describe qué hace la flota sino qué debe cumplirse. Actúa como la especificación contra la cual el compilador contrasta el plan que reconstruyó.

Todas sus cláusulas son opcionales, y las que admiten repetición, como la exclusión de zonas, pueden aparecer varias veces.

```
constraints {
  min_separation 8 [m];
  battery_reserve 20 [%];
  exclude LKP;
  require coverage of AOI >= 95 [%];
}
```

4.9. Verificación

El compilador realiza la totalidad de las verificaciones antes de emitir los artefactos de salida. Ante cualquier falla la compilación se aborta. Las verificaciones se organizan en cuatro niveles:

Sobre el AST, los verificaciones de bajo costo computacional que no requieren simulación aquí caen las declaraciones previas a su uso, corrección dimensional de las unidades, formaciones con tantas posiciones como aeronaves, maniobras contenidas dentro de la ventana de misión, y ausencia de maniobras simultáneas sobre una misma flota.

Sobre las trayectorias simulada, con las trayectorias ya muestreadas, el compilador las recorre instante por instante y controla que ninguna aeronave supere ni su velocidad máxima ni su aceleración máxima, que todas permanezcan dentro de las zonas autorizadas, que ninguna quede sin enlace con la estación y que en ningún momento dos aeronaves estén más cerca que la separación mínima.

Para verificar la energía se integra un modelo de consumo diferenciado por fase de vuelo (ascenso, operacion y estacionario), y se exige que cada aeronave aterrice con la reserva de batería declarada.

Por último en cuanto a cobertura, se calcula el ancho de franja observado por el sensor según la altitud de vuelo, y se reporta el porcentaje de cada zona que resulta efectivamente observado, descontando la superficie de las zonas excluidas que intersectan el área de barrido.

Los mensajes de error indican en todos los casos la aeronave involucrada, y cuando es posible, el instante y la magnitud de la violación.

4.10. Artefactos de salida

Una vez que la misión pasa todas las verificaciones, el compilador emite tres artefactos, cada uno pensado para un consumidor y un momento distinto del flujo de trabajo:

Plan de misión (JSON): es la salida operativa y contiene los waypoints absolutos de cada aeronave con sus marcas temporales, es decir, la trayectoria completa ya resuelta.

Previsualización (HTML + SVG): es la salida de inspección, dibuja sobre el plano todos los elementos espaciales de la misión, las zonas (autorizadas y de exclusión), las formaciones, la traza que recorre cada aeronave y el mapa de cobertura logrado por los sensores. Su función es que el operador confirme visualmente que la misión hace lo que él pretendía y detecte a ojo cosas que los números no muestran tan fácil, como una traza que pasa raspando una zona prohibida.

Reporte (LaTeX): por último la salida documental. Incluye el diagrama de fases de la misión, la tabla de consumo energético por aeronave y el porcentaje de cobertura obtenido sobre cada zona. No está pensado para operar ni para inspeccionar, sino para archivo y auditoría posterior.

Al combinar las tres salidas del sistema se busca lograr una verificación que produce una vista para la máquina, una para el ojo del operador y una para el registro institucional. Aportando una vista general sobre una misma misión abarcando las partes fundamentales de la misma.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que declare un tipo de dron con todos sus campos: autonomía, velocidad máxima, aceleración máxima, sensor de carga útil y alcance de enlace.
- (II). Un programa que declare un tipo de dron omitiendo los campos opcionales, conservando sólo autonomía y velocidad máxima.
- (III). Un programa que componga una flota heterogénea a partir de dos tipos distintos.
- (IV). Un programa que dirija una maniobra a una sub flota mediante notación de punto.
- (V). Un programa que declare una zona circular indicando centro y radio.
- (VI). Un programa que declare una estación base con su posición y su altitud.
- (VII). Un programa que declare una formación de tipo grilla y ordene a una flota de cardinalidad coincidente adoptarla.
- (VIII). Un programa que declare una formación de tipo círculo y otra de tipo línea, que se acomodan a la cantidad de aeronaves de la flota.
- (IX). Un programa que declare una formación en el punto de uso, sin nombrarla previamente.
- (X). Un programa que especifique una misión mínima con ventana temporal, despegue y aterrizaje.

(XI). Un programa cuyas maniobras se encadenen implícitamente, sin bloques de composición explícitos.

(XII). Un programa que anide un bloque secuencial dentro de uno paralelo.

(XIII). Un programa que ordene un cambio de formación con duración explícita.

(XIV). Un programa que ejecute un barrido con patrón lawnmower sobre una zona.

(XV). Un programa que ordene una órbita sobre un punto de interés durante un intervalo, y una espera en vuelo estacionario.

(XVI). Un programa que ordene el retorno de todas las aeronaves a la estación con una fecha límite.

(XVII). Un programa que ejecute un barrido sobre una zona que contiene una zona excluida, verificando que el patrón la contornee.

(XVIII). Un programa que declare una zona poligonal convexa de tres vértices.

(XIX). Un programa que declare un bloque de restricciones con separación mínima, reserva de batería, exclusión de zona y umbral de cobertura.

Además, los siguientes casos de prueba de **rechazo**:

(I). Un programa con un literal numérico malformado.

(II). Un programa que utilice una unidad que el lenguaje no reconoce.

(III). Un programa que contenga un carácter ajeno al alfabeto del lenguaje.

(IV). Un programa que utilice una palabra reservada como identificador.

(V). Un programa donde una magnitud física se declare sin su unidad.

(VI). Un programa que declare un polígono con menos de tres vértices.

(VII). Un programa que componga una flota a partir de un tipo de dron no declarado.

(VIII). Un programa cuya formación de tipo grilla exija más posiciones que aeronaves tiene la flota que la adopta.

(IX). Un programa que declare un polígono cóncavo.

(X). Un programa cuya maniobra quede fuera de la ventana temporal de la misión.

(XI). Un programa que ordene dos maniobras simultáneas sobre la misma subflota dentro de un bloque paralelo.

(XII). Un programa cuyas trayectorias violen la separación mínima entre aeronaves.

(XIII). Un programa cuyo cambio de formación exija superar la velocidad máxima del tipo de aeronave.

(XIV). Un programa cuya misión agote la autonomía de una aeronave antes del retorno a base.

(XV). Un programa en el que una aeronave exceda su alcance de enlace respecto de la estación.

(XVI). Un programa que asigne a un parámetro una magnitud con una unidad dimensionalmente incorrecta.

6. Ejemplos

Se declaran dos tipos de dron, cada uno con su autonomía, sus límites cinemáticos y el sensor que transporta, y a partir de ellos se compone una flota heterogénea.

// Declarar un tipo de dron con autonomía, límites cinemáticos y sensor térmico:

```
drone_type Quad {
    endurance 35 [min];
    max_speed 15 [m/s];
    max_accel 3 [m/s^2];
    payload thermal_camera resolution 640 [px] fov 57 [deg];
    link_range 1000 [m];
}
```

// Declarar un tipo de ala fija con mayor autonomía y sensor RGB:

```
drone_type FixedWing {
    endurance 90 [min];
    max_speed 25 [m/s];
    max_accel 2 [m/s^2];
    payload rgb_camera resolution 4000 [px] fov 70 [deg];
    //link_range 1500 [m]; opcional
}
```

// Componer una flota heterogénea a partir de los tipos anteriores:

fleet Alpha = 3 of Quad + 1 of FixedWing;

Se define el espacio de operación: una zona autorizada poligonal, una zona de exclusión circular, la estación base desde la que despegan y aterrizan las aeronaves, y las restricciones de seguridad que el compilador verifica sobre las trayectorias. Toda magnitud lleva su unidad (con la excepción de las coordenadas que se interpretan en metros) , de modo que el compilador pueda comprobar la corrección dimensional.

// Zona autorizada poligonal, definida por sus vértices:

zone Campo = polygon [(0,0), (800,0), (800,500), (0,500)];

// Zona circular alrededor de una antena (excluida en el bloque de restricciones):

zone Antena = circle center (600, 400) radius 60 [m];

// Estación base: punto de despegue y aterrizaje:

station Base = point (10, 10) altitude 0 [m];

// Restricciones globales que el compilador verifica sobre las trayectorias:

```
constraints {
    min_separation 20 [m];
    battery_reserve 20 [%];
    exclude Antena;
}
```

Se especifica una misión completa: dos flotas despegan de forma escalonada desde la estación, una de ellas adopta una formación en línea y ejecuta un barrido de superficie —cuyo patrón contournea la zona excluida de la antena, declarada en el ejemplo anterior— mientras la

otra órbita un punto de interés en simultáneo, y todas las aeronaves regresan a la base antes del cierre de la ventana temporal.

```

fleet Beta= 2 of Quad;
zone PDI = circle center (200, 300) radius 10 [m];
formation Peine = line spacing 30 [m];

mission Relevamiento window 0 [s] .. 1500 [s] {
    // Despegue escalonado de ambas flotas desde la estación:
    Alpha takeoff from Base at 0 [s];
    Beta takeoff from Base at 30 [s];

    // Adoptar formación de barrido antes de trabajar:
    Alpha morph to Peine over 20 [s];

    // Dos maniobras simultáneas sobre el mismo terreno:
    parallel {
        Alpha sweep Campo with lawnmower spacing 40 [m] altitude 60 [m];
        Beta orbit PDI radius 80 [m] altitude 90 [m] for 600 [s];
    }

    // Retorno de ambas flotas antes del cierre de la ventana:
    all return to Base before 1400 [s];
}

```